

Java 7, 8 & 9 New Features



Trainee handout with runnable example programs

Part 1 - Java 7 (17 topics)

Part 2 - Java 8 (25 topics)

Part 3 - Java 9 (9 topics)

Contents

How to Use This Handout	4
Part 1 - Java 7 New Features	5
1.1 Try-with-Resources	6
1.2 Suppressed Exceptions	7
1.3 Catching Multiple Exceptions	8
1.4 Rethrowing with Precise Type Checking	8
1.5 Easier Exception Handling for Reflection	9
1.6 Objects Class and Null Checks	10
1.7 close() Method in URLClassLoader	12
1.8 @SafeVarargs Annotation	12
1.9 Files and Directories (NIO.2)	13
1.10 WatchService	14
1.11 Binary Literals	15
1.12 Underscores in Numeric Literals	16
1.13 String in switch Statement	17
1.14 Type Inference - Diamond Operator	18
1.15 JDBC Improvements (JDBC 4.1)	18
1.16 Garbage-First (G1) Collector	19
1.17 Fork/Join Framework	20
Part 2 - Java 8 New Features	23
2.1 Default Methods in Interfaces	24
2.2 Interface Defaults vs Abstract Classes	25
2.3 Static Methods in Interfaces	25
2.4 Optional - Dealing with Nulls	26
2.5 Lambda Expressions	27
2.6 Anonymous Inner Class vs Lambda	28
2.7 Functional Interfaces	29
2.8 Predicate	30
2.9 Function	30
2.10 UnaryOperator and BinaryOperator	31
2.11 Consumer and Supplier	32
2.12 Bi-Functional Interfaces	33

2.13 Primitive Functional Interfaces	34
2.14 Method References	35
2.15 Constructor References	36
2.16 Streams API - Basics	37
2.17 Streams API - Operations and Collectors	38
2.18 Collections vs Streams	40
2.19 New Date and Time API - Local Types	40
2.20 Time Zones, Instant, Duration and Formatting	42
2.21 CompletableFuture	43
2.22 Map Enhancements	44
2.23 Other Miscellaneous Updates	45

Part 3 - Java 9 New Features 48

3.1 Private Methods in Interfaces	49
3.2 Try-with-Resources Improvement	50
3.3 Collection Factory Methods	50
3.4 Stream API Enhancements	51
3.5 Optional Enhancements	52
3.6 Collectors Enhancements	53
3.7 Process API Improvements	54
3.8 Arrays Enhancements	55
3.9 Other Miscellaneous Updates	56

How to Use This Handout

Each topic has a short explanation, key points to remember, a complete example program, and the actual output of that program. Import statements are left out of the printed code to save space - they are in the project files.

Project layout. The companion Maven project `java-new-features-training` has three modules - `java7`, `java8` and `java9` - with every class under the package `com.training.<version>`. Every class has its own `main` method.

Running. Import the root folder into Eclipse or IntelliJ as an existing Maven project and run any class, or from the command line: `mvn compile` then `java -cp java8/target/classes com.training.java8.StreamOperationsDemo`. Any JDK 11 or newer works; the output shown was produced on JDK 21.

Tip for practice. Before looking at the output, predict what each program will print. Then change one thing - a predicate, a collector, a date - and predict again.

Part 1 - Java 7 New Features

Released in July 2011, Java 7 (Project Coin + NIO.2) focused on developer productivity: less boilerplate in exception handling and generics, a modern file-system API, and a framework for parallel divide-and-conquer work.

01	Try-with-Resources	TryWithResourcesDemo
02	Suppressed Exceptions	SuppressedExceptionsDemo
03	Catching Multiple Exceptions	MultiCatchDemo
04	Rethrowing with Precise Type Checking	PreciseRethrowDemo
05	Easier Exception Handling for Reflection	ReflectiveOperationDemo
06	Objects Class and Null Checks	ObjectsUtilityDemo
07	close() Method in URLClassLoader	UrlClassLoaderCloseDemo
08	@SafeVarargs Annotation	SafeVarargsDemo
09	Files and Directories (NIO.2)	NioFilesDemo
10	WatchService	WatchServiceDemo
11	Binary Literals	BinaryLiteralsDemo
12	Underscores in Numeric Literals	UnderscoreLiteralsDemo
13	String in switch Statement	StringSwitchDemo
14	Type Inference - Diamond Operator	DiamondOperatorDemo
15	JDBC Improvements (JDBC 4.1)	JdbcImprovementsDemo
16	Garbage-First (G1) Collector	G1CollectorInfoDemo
17	Fork/Join Framework	ForkJoinDemo

1.1 Try-with-Resources

Any object that implements **AutoCloseable** can be declared in the parentheses of a try statement. The JVM calls **close()** automatically when the block ends, whether it ends normally or with an exception, so there is no need for a finally block.

KEY POINTS

- Resources are closed in the reverse order of declaration.
- Multiple resources are separated by semicolons.
- catch and finally blocks are still allowed and run after the resources are closed.
- JDBC, streams, readers/writers, Scanner, sockets and channels all implement AutoCloseable.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/TryWithResourcesDemo.java

```
/** Java 7: try-with-resources closes every AutoCloseable automatically. */
public class TryWithResourcesDemo {

    static class DbConnection implements AutoCloseable {
        private final String name;
        DbConnection(String name) { this.name = name; System.out.println("Open  " + name); }
        void query(String sql)    { System.out.println("Run   " + sql + " on " + name); }
        @Override public void close() { System.out.println("Close " + name); }
    }

    public static void main(String[] args) throws IOException {
        // Resources are closed in REVERSE order of declaration
        try (DbConnection primary = new DbConnection("primary-db");
            DbConnection audit = new DbConnection("audit-db")) {
            primary.query("SELECT * FROM accounts");
            audit.query("INSERT INTO audit_log ...");
        }

        // Works with every JDK class that implements AutoCloseable
        try (BufferedReader reader = new BufferedReader(new StringReader("line-1\nline-2"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println("Read: " + line);
            }
        }
    }
}
```

OUTPUT

```
Open  primary-db
Open  audit-db
Run   SELECT * FROM accounts on primary-db
Run   INSERT INTO audit_log ... on audit-db
Close audit-db
Close primary-db
Read: line-1
Read: line-2
```

1.2 Suppressed Exceptions

If the try block throws and then **close()** also throws, the first exception is kept as the primary one and the close-time exception is attached to it as a *suppressed* exception, so no failure information is lost.

KEY POINTS

- Read them with `Throwable.getSuppressed()`.
- Add your own with `Throwable.addSuppressed()` - useful for batch jobs that collect errors.
- Before Java 7 the exception from finally/close silently replaced the original one.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/SuppressedExceptionsDemo.java

```
/** Java 7: exceptions thrown from close() are attached as "suppressed" exceptions. */
public class SuppressedExceptionsDemo {

    static class FaultyResource implements AutoCloseable {
        void process() { throw new IllegalStateException("Failure while processing"); }
        @Override public void close() { throw new RuntimeException("Failure while closing"); }
    }

    public static void main(String[] args) {
        try (FaultyResource resource = new FaultyResource()) {
            resource.process();
        } catch (Exception e) {
            System.out.println("Primary   : " + e.getMessage());
            for (Throwable s : e.getSuppressed()) {
                System.out.println("Suppressed: " + s.getMessage());
            }
        }

        // We can also add suppressed exceptions manually
        Exception main = new Exception("Batch job failed");
        main.addSuppressed(new Exception("Record 17 invalid"));
        main.addSuppressed(new Exception("Record 42 invalid"));
        System.out.println(main.getMessage() + " -> suppressed count = "
            + main.getSuppressed().length);
    }
}
```

OUTPUT

```
Primary   : Failure while processing
Suppressed: Failure while closing
Batch job failed -> suppressed count = 2
```

1.3 Catching Multiple Exceptions

A single catch block can handle several exception types separated by `|`. This removes duplicated catch blocks that contained the same handling code.

KEY POINTS

- The catch parameter is implicitly final.
- Types in a multi-catch must not be subclasses of each other (compile error).
- The generated bytecode is smaller than equivalent separate catch blocks.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/MultiCatchDemo.java

```
/** Java 7: catch several unrelated exception types in a single catch block. */
public class MultiCatchDemo {

    static void parseAndDivide(String a, String b) {
        try {
            int x = Integer.parseInt(a);
            int y = Integer.parseInt(b);
            System.out.println(a + " / " + b + " = " + (x / y));
        } catch (NumberFormatException | ArithmeticException e) {
            // 'e' is implicitly final here - it cannot be reassigned
            System.out.println("Bad input (" + e.getClass().getSimpleName() + "): "
                               + e.getMessage());
        }
    }

    public static void main(String[] args) {
        parseAndDivide("100", "4");
        parseAndDivide("100", "abc");
        parseAndDivide("100", "0");
    }
}
```

OUTPUT

```
100 / 4 = 25
Bad input (NumberFormatException): For input string: "abc"
Bad input (ArithmeticException): / by zero
```

1.4 Rethrowing with Precise Type Checking

When you catch a broad type such as **Exception** and rethrow it unchanged, the compiler now analyses the try block and allows the method to declare only the specific checked exceptions that can really occur.

KEY POINTS

- Lets you log-and-rethrow without widening the method signature to 'throws Exception'.
- Works only if the catch parameter is effectively final (not reassigned).

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/PreciseRethrowDemo.java

```
/**
 * Java 7: when a caught Exception is rethrown unchanged, the compiler knows
 * exactly which checked exceptions the try block can throw.
 */
public class PreciseRethrowDemo {

    static void load(int source) throws FileNotFoundException, SQLException {
        if (source == 1) throw new FileNotFoundException("config.yml missing");
        if (source == 2) throw new SQLException("DB unreachable");
        System.out.println("Loaded source " + source);
    }

    // Before Java 7 this method would have to declare "throws Exception"
    static void loadWithLogging(int source) throws FileNotFoundException, SQLException {
        try {
            load(source);
        } catch (Exception e) {
            System.out.println("LOG: " + e.getMessage());
            throw e; // precise rethrow: only the two checked types can escape
        }
    }

    public static void main(String[] args) {
        for (int source = 0; source <= 2; source++) {
            try {
                loadWithLogging(source);
            } catch (FileNotFoundException | SQLException e) {
                System.out.println("Caller handled " + e.getClass().getSimpleName());
            }
        }
    }
}
```

OUTPUT

```
Loaded source 0
LOG: config.yml missing
Caller handled FileNotFoundException
LOG: DB unreachable
Caller handled SQLException
```

1.5 Easier Exception Handling for Reflection

ReflectiveOperationException is a new common superclass of `ClassNotFoundException`, `NoSuchMethodException`, `NoSuchFieldException`, `IllegalAccessException`, `InstantiationException` and `InvocationTargetException`.

KEY POINTS

- One catch block replaces up to six.
- Frameworks that use reflection heavily (DI containers, ORMs, test runners) benefit most.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/ReflectiveOperationDemo.java

```
/** Java 7: ReflectiveOperationException is a common parent for reflection errors. */
public class ReflectiveOperationDemo {

    public String greet(String name) { return "Hello, " + name; }

    static void invoke(String className, String methodName) {
        try {
            Class<?> type = Class.forName(className);
            Object target = type.getDeclaredConstructor().newInstance();
            Method method = type.getMethod(methodName, String.class);
            System.out.println(method.invoke(target, "Trainee"));
        } catch (ReflectiveOperationException e) {
            // One catch replaces ClassNotFoundException, NoSuchMethodException,
            // IllegalAccessException and InvocationTargetException exceptions
            System.out.println("Reflection failed: " + e.getClass().getSimpleName());
        }
    }

    public static void main(String[] args) {
        invoke("com.training.java7.ReflectiveOperationDemo", "greet");
        invoke("com.training.java7.ReflectiveOperationDemo", "unknownMethod");
        invoke("com.training.DoesNotExist", "greet");
    }
}
```

OUTPUT

```
Hello, Trainee
Reflection failed: NoSuchMethodException
Reflection failed: ClassNotFoundException
```

1.6 Objects Class and Null Checks

java.util.Objects is a utility class with null-safe versions of common operations: equals, hashCode, hash, toString and requireNonNull.

KEY POINTS

- `requireNonNull(obj, message)` validates constructor/method arguments and fails fast.
- `Objects.equals(a, b)` is safe when either argument is null.
- `Objects.hash(fields...)` makes writing `hashCode()` a one-liner.
- Java 8 added `isNull/nonNull`, Java 9 added `requireNonNullElse` (see Part 3).

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/ObjectsUtilityDemo.java

```
/** Java 7: java.util.Objects offers null-safe helper methods. */
public class ObjectsUtilityDemo {

    static class Customer {
        private final String id;
        private final String email;

        Customer(String id, String email) {
            // Fail fast with a clear message instead of a later NullPointerException
            this.id = Objects.requireNonNull(id, "Customer id must not be null");
            this.email = email;
        }

        @Override public boolean equals(Object o) {
            if (!(o instanceof Customer)) return false;
            Customer other = (Customer) o;
            return Objects.equals(id, other.id) && Objects.equals(email, other.email);
        }

        @Override public int hashCode() { return Objects.hash(id, email); }
        @Override public String toString() {
            return id + " <" + Objects.toString(email, "no-email") + ">";
        }
    }

    public static void main(String[] args) {
        Customer a = new Customer("C001", null);
        Customer b = new Customer("C001", null);
        System.out.println("a = " + a);
        System.out.println("a.equals(b)      : " + a.equals(b));
        System.out.println("same hashCode    : " + (a.hashCode() == b.hashCode()));
        System.out.println("Objects.equals(null, null): " + Objects.equals(null, null));
        try {
            new Customer(null, "x@y.com");
        } catch (NullPointerException e) {
            System.out.println("NPE message      : " + e.getMessage());
        }
    }
}
```

OUTPUT

```
a = C001 <no-email>
a.equals(b)      : true
same hashCode    : true
Objects.equals(null, null): true
NPE message      : Customer id must not be null
```

1.7 close() Method in URLClassLoader

URLClassLoader now implements **Closeable**. Closing it releases the jar and directory handles it opened, so plugin jars can be replaced or deleted without restarting the JVM.

KEY POINTS

- Important on Windows, where open files cannot be deleted.
- Can be used in try-with-resources.
- Classes already loaded keep working after close().

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/UrlClassLoaderCloseDemo.java

```
/** Java 7: URLClassLoader implements Closeable, so it releases jar/file handles. */
public class UrlClassLoaderCloseDemo {

    public static void main(String[] args) throws IOException {
        Path pluginDir = Files.createTempDirectory("plugins");
        URL[] urls = { pluginDir.toUri().toURL() };

        try (URLClassLoader loader = new URLClassLoader(urls)) {
            System.out.println("Loader created for: " + loader.getURLs()[0]);
            // loader.loadClass("com.acme.Plugin") would load plugin classes here
        } // close() releases open files - the plugin folder can now be replaced

        Files.delete(pluginDir);
        System.out.println("Plugin folder deleted: " + !Files.exists(pluginDir));
    }
}
```

OUTPUT

```
Loader created for: file:/tmp/plugins12956267292719023799/
Plugin folder deleted: true
```

1.8 @SafeVarargs Annotation

Generic varargs methods create a generic array internally, which triggers an 'unchecked / possible heap pollution' warning at every call site. If the method only reads from the array, annotate it with **@SafeVarargs** to suppress the warning.

KEY POINTS

- Allowed on static and final methods and constructors (Java 7), private methods (Java 9).
- It is a promise by the author - the compiler does not verify it.
- JDK examples: Arrays.asList, List.of, Stream.of, EnumSet.of.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/SafeVarargsDemo.java

```
/** Java 7: @SafeVarargs silences "unchecked generic array" warnings for safe methods. */
public class SafeVarargsDemo {

    // Without the annotation, callers get a "possible heap pollution" warning
    @SafeVarargs
    static <T> List<T> combine(List<? extends T>... lists) {
        List<T> result = new ArrayList<>();
        for (List<? extends T> list : lists) {
            result.addAll(list);    // we only READ from the varargs array - safe
        }
        return result;
    }

    public static void main(String[] args) {
        List<String> branchA = Arrays.asList("Chennai", "Madurai");
        List<String> branchB = Arrays.asList("Bangalore");
        List<String> branchC = Arrays.asList("Kochi", "Hyderabad");
        System.out.println(combine(branchA, branchB, branchC));
    }
}
```

OUTPUT

[Chennai, Madurai, Bangalore, Kochi, Hyderabad]

1.9 Files and Directories (NIO.2)

The **java.nio.file** package introduces **Path** (a location), **Paths** (factory) and **Files** (static operations). It replaces most uses of `java.io.File` with clearer methods and proper exceptions.

KEY POINTS

- `Files.readAllLines` / `write` / `copy` / `move` / `delete` / `exists` / `size` / `createDirectories`.
- `DirectoryStream` lists folders with glob patterns such as `*.txt`.
- Errors throw meaningful exceptions (`NoSuchFileException` etc.) instead of returning false.
- Supports symbolic links, file attributes and pluggable file systems (e.g. zip).

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/NioFilesDemo.java

```
/** Java 7 (NIO.2): Path, Paths and Files replace most uses of java.io.File. */
public class NioFilesDemo {

    public static void main(String[] args) throws IOException {
        Path base = Files.createTempDirectory("nio-demo");
        Path reports = base.resolve("reports");
        Files.createDirectories(reports);

        Path path = Paths.get(reports.toString(), "q1.txt");
        System.out.println("File name : " + path.getFileName());
        System.out.println("Parent    : " + path.getParent().getFileName());

        List<String> lines = Arrays.asList("Revenue: 120", "Expenses: 80");
        Files.write(path, lines, StandardCharsets.UTF_8);
        System.out.println("Exists    : " + Files.exists(path)
            + ", size = " + Files.size(path) + " bytes");

        Path backup = reports.resolve("q1-backup.txt");
        Files.copy(path, backup, StandardCopyOption.REPLACE_EXISTING);
        System.out.println("Read back : " + Files.readAllLines(backup, StandardCharsets.UTF_8));

        try (DirectoryStream<Path> stream = Files.newDirectoryStream(reports, "*.txt")) {
            for (Path p : stream) System.out.println("Listed    : " + p.getFileName());
        }

        Files.delete(backup);
        Files.delete(path);
        Files.delete(reports);
        Files.delete(base);
        System.out.println("Cleaned up: " + !Files.exists(base));
    }
}
```

OUTPUT

```
File name : q1.txt
Parent    : reports
Exists    : true, size = 26 bytes
Read back : [Revenue: 120, Expenses: 80]
Listed    : q1.txt
Listed    : q1-backup.txt
Cleaned up: true
```

1.10 WatchService

A **WatchService** lets a program register a directory and receive events when files are created, modified or deleted, instead of repeatedly polling the folder.

KEY POINTS

- Register with ENTRY_CREATE, ENTRY_MODIFY and ENTRY_DELETE.
- take() blocks until an event arrives; poll(timeout) waits for a limited time.
- Always call key.reset() after processing, or you stop receiving events.
- Typical uses: hot-reloading config, file-drop integrations, auto-import folders.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/WatchServiceDemo.java

```
/** Java 7: WatchService receives file-system events instead of polling a folder. */
public class WatchServiceDemo {

    public static void main(String[] args) throws Exception {
        final Path inbox = Files.createTempDirectory("inbox");

        try (WatchService watcher = FileSystems.getDefault().newWatchService()) {
            inbox.register(watcher, StandardWatchEventKinds.ENTRY_CREATE,
                StandardWatchEventKinds.ENTRY_DELETE);

            // Simulate another process dropping a file into the folder
            new Thread(new Runnable() {
                public void run() {
                    try {
                        Thread.sleep(500);
                        Path f = Files.createFile(inbox.resolve("invoice-101.csv"));
                        Thread.sleep(500);
                        Files.delete(f);
                    } catch (Exception ignored) { }
                }
            }).start();

            int received = 0;
            while (received < 2) {
                WatchKey key = watcher.poll(10, TimeUnit.SECONDS);
                if (key == null) break; // timed out
                for (WatchEvent<?> event : key.pollEvents()) {
                    System.out.println(event.kind().name() + " -> " + event.context());
                    received++;
                }
                key.reset(); // keep receiving events
            }
            Files.delete(inbox);
        }
    }
}
```

OUTPUT

```
ENTRY_CREATE -> invoice-101.csv
ENTRY_DELETE -> invoice-101.csv
```

1.11 Binary Literals

Integer types (byte, short, int, long) can be written in binary using the **0b** or **0B** prefix. This is handy for bit masks, flags and protocol values.

KEY POINTS

- `0b101 == 5`.
- Combine with underscores for readability: `0b1111_0000`.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/BinaryLiteralsDemo.java

```
/** Java 7: integer literals can be written in binary with the 0b / 0B prefix. */
public class BinaryLiteralsDemo {

    // Permission flags are much easier to read in binary
    static final int READ    = 0b001;
    static final int WRITE   = 0b010;
    static final int EXECUTE = 0b100;

    public static void main(String[] args) {
        int permissions = READ | WRITE;
        System.out.println("Permissions (decimal): " + permissions);
        System.out.println("Permissions (binary) : " + Integer.toBinaryString(permissions));
        System.out.println("Can write?      " + ((permissions & WRITE) != 0));
        System.out.println("Can execute?   " + ((permissions & EXECUTE) != 0));

        byte mask = (byte) 0b1111_0000;
        System.out.println("byte 0b1111_0000 = " + mask);
    }
}
```

OUTPUT

```
Permissions (decimal): 3
Permissions (binary) : 11
Can write?      true
Can execute?   false
byte 0b1111_0000 = -16
```

1.12 Underscores in Numeric Literals

Underscores can be placed between digits of any numeric literal to group them. They are ignored by the compiler and exist only to improve readability.

KEY POINTS

- Not allowed at the start or end of a number, next to a decimal point, next to the L/F/D suffix, or right after 0x / 0b.
- Any grouping works - e.g. Indian lakh grouping 25_00_000.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/UnderscoreLiteralsDemo.java

```
/** Java 7: underscores between digits make large numeric literals readable. */
public class UnderscoreLiteralsDemo {

    public static void main(String[] args) {
        long loanAmount    = 25_00_000L;           // Indian grouping: 25 lakh
        long population     = 1_428_000_000L;       // Western grouping
        int cardPrefix      = 4111_1111;
        double interest     = 8_5.25;              // allowed, though unusual
        int hexColor        = 0xFF_A5_00;
        int flags           = 0b1010_0101;

        System.out.println("Loan          : " + loanAmount);
        System.out.println("Population : " + population);
        System.out.println("Card prefix: " + cardPrefix);
        System.out.println("Interest  : " + interest);
        System.out.println("Hex colour : " + hexColor);
        System.out.println("Flags      : " + flags);

        // Not allowed: _100, 100_, 1._5, 0xFF, 10_L -> compile errors
    }
}
```

OUTPUT

```
Loan      : 2500000
Population : 1428000000
Card prefix: 41111111
Interest  : 85.25
Hex colour : 16753920
Flags     : 165
```

1.13 String in switch Statement

switch statements can use **String** values in case labels. The comparison uses `String.equals()`, so it is case-sensitive.

KEY POINTS

- Switching on a null String throws `NullPointerException` - check for null first.
- The compiler implements it using `hashCode()` followed by `equals()`, so it is efficient.

EXAMPLE PROGRAM

`java7/src/main/java/com/training/java7/StringSwitchDemo.java`

```
/** Java 7: switch statements can use String values. */
public class StringSwitchDemo {

    static double deliveryCharge(String city) {
        switch (city.toLowerCase()) {          // comparison uses String.equals()
            case "bangalore":
            case "chennai":
                return 40.0;
            case "mumbai":
                return 60.0;
            default:
                return 99.0;
        }
    }

    public static void main(String[] args) {
        String[] cities = { "Bangalore", "CHENNAI", "Mumbai", "Jaipur" };
        for (String city : cities) {
            System.out.println(city + " -> Rs." + deliveryCharge(city));
        }
        // Note: switch on a null String throws NullPointerException
    }
}
```

OUTPUT

```
Bangalore -> Rs.40.0
CHENNAI -> Rs.40.0
Mumbai -> Rs.60.0
Jaipur -> Rs.99.0
```

1.14 Type Inference - Diamond Operator

When creating a generic object you can write empty angle brackets <> and the compiler infers the type arguments from the variable declaration.

KEY POINTS

- Removes repetition in declarations like Map<String, List<Integer>>.
- Java 7/8 do not allow <> with anonymous inner classes - Java 9 does.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/DiamondOperatorDemo.java

```
/** Java 7: the diamond operator <> lets the compiler infer generic type arguments. */
public class DiamondOperatorDemo {

    public static void main(String[] args) {
        // Before Java 7:
        Map<String, List<Integer>> oldStyle = new HashMap<String, List<Integer>>();

        // Java 7 onwards:
        Map<String, List<Integer>> marksByStudent = new HashMap<>();
        List<Integer> ravi = new ArrayList<>();
        ravi.add(88);
        ravi.add(92);
        marksByStudent.put("Ravi", ravi);
        marksByStudent.put("Anu", new ArrayList<>(ravi));

        System.out.println(marksByStudent);
        System.out.println("oldStyle is empty: " + oldStyle.isEmpty());
    }
}
```

OUTPUT

```
{Ravi=[88, 92], Anu=[88, 92]}
oldStyle is empty: true
```

1.15 JDBC Improvements (JDBC 4.1)

Connection, Statement and ResultSet now implement AutoCloseable, so they can be managed by try-with-resources. **RowSetProvider** and **RowSetFactory** create RowSet objects without referring to vendor-specific implementation classes.

KEY POINTS

- No more nested finally blocks with null checks to close JDBC objects.
- Pass a JDBC URL as a program argument to run the query part against a real database.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/JdbcImprovementsDemo.java

```
/**
 * Java 7 (JDBC 4.1): Connection, Statement and ResultSet are AutoCloseable,
 * and RowSetProvider creates RowSets without vendor-specific classes.
 */
public class JdbcImprovementsDemo {

    // Pass a JDBC URL as the first argument to run this against a real database
    static void printAccounts(String url) throws SQLException {
        String sql = "SELECT id, holder FROM accounts WHERE balance > ?";
        try (Connection con = DriverManager.getConnection(url);
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setDouble(1, 10_000);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    System.out.println(rs.getInt("id") + " " + rs.getString("holder"));
                }
            }
        } // connection, statement and result set are all closed here
    }

    public static void main(String[] args) throws SQLException {
        RowSetFactory factory = RowSetProvider.newFactory();
        try (CachedRowSet rowSet = factory.createCachedRowSet()) {
            System.out.println("RowSetFactory : " + factory.getClass().getSimpleName());
            System.out.println("CachedRowSet : " + rowSet.getClass().getSimpleName());
        }
        if (args.length > 0) {
            printAccounts(args[0]);
        } else {
            System.out.println("No JDBC URL given - skipping database query.");
        }
    }
}
```

OUTPUT

```
RowSetFactory : RowSetFactoryImpl
CachedRowSet : CachedRowSetImpl
No JDBC URL given - skipping database query.
```

1.16 Garbage-First (G1) Collector

G1 became a fully supported collector in Java 7 update 4. It divides the heap into many equal-sized regions and collects the regions with the most garbage first, aiming for short, predictable pause times on large heaps.

KEY POINTS

- Enable with -XX:+UseG1GC (default collector since Java 9).
- Set a pause-time goal with -XX:MaxGCPauseMillis=200.
- Replaced the CMS collector, which was removed in Java 14.
- Output depends on the JVM version and collector in use.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/G1CollectorInfoDemo.java

```
/**
 * Java 7u4: the Garbage-First (G1) collector became fully supported.
 * Run with: java -XX:+UseG1GC -Xmx256m com.training.java7.G1CollectorInfoDemo
 * (G1 is already the default collector from Java 9 onwards.)
 */
public class G1CollectorInfoDemo {

    public static void main(String[] args) {
        List<byte[]> garbage = new ArrayList<>();
        for (int i = 0; i < 2_000; i++) {
            garbage.add(new byte[10_000]);
            if (garbage.size() > 200) garbage.clear();    // make objects unreachable
        }
        System.gc();

        for (GarbageCollectorMXBean gc : ManagementFactory.getGarbageCollectorMXBeans()) {
            System.out.println("Collector: " + gc.getName()
                               + " | collections run: " + (gc.getCollectionCount() > 0 ? "yes" : "no"));
        }
    }
}
```

OUTPUT

```
Collector: G1 Young Generation | collections run: yes
Collector: G1 Concurrent GC | collections run: no
Collector: G1 Old Generation | collections run: yes
```

1.17 Fork/Join Framework

The Fork/Join framework (**ForkJoinPool**, **RecursiveTask**, **RecursiveAction**) is designed for divide-and-conquer problems. A task splits itself into smaller subtasks (fork), runs them in parallel, and combines their results (join).

KEY POINTS

- `RecursiveTask<V>` returns a value; `RecursiveAction` returns nothing.
- Idle worker threads 'steal' queued work from busy threads (work-stealing).
- Pick a sensible threshold - splitting too finely adds overhead.
- Java 8 parallel streams run on the common `ForkJoinPool`.
- The parallelism value depends on the number of CPU cores.

EXAMPLE PROGRAM

java7/src/main/java/com/training/java7/ForkJoinDemo.java

```
/** Java 7: Fork/Join splits a big task into sub-tasks that run in parallel. */
public class ForkJoinDemo {

    static class SumTask extends RecursiveTask<Long> {
        private static final long serialVersionUID = 1L;
        private static final int THRESHOLD = 10_000;
        private final long[] data;
        private final int from, to;

        SumTask(long[] data, int from, int to) {
            this.data = data; this.from = from; this.to = to;
        }

        @Override
        protected Long compute() {
            if (to - from <= THRESHOLD) {                // small enough: do it directly
                long sum = 0;
                for (int i = from; i < to; i++) sum += data[i];
                return sum;
            }
            int mid = (from + to) >> 1;
            SumTask left = new SumTask(data, from, mid);
            SumTask right = new SumTask(data, mid, to);
            left.fork();                                   // run left half asynchronously
            long rightResult = right.compute();           // compute right half here
            return left.join() + rightResult;             // wait for left and combine
        }
    }

    public static void main(String[] args) {
        long[] transactions = new long[1_000_000];
        for (int i = 0; i < transactions.length; i++) transactions[i] = i + 1;

        ForkJoinPool pool = new ForkJoinPool();
        long total = pool.invoke(new SumTask(transactions, 0, transactions.length));
        System.out.println("Parallelism : " + pool.getParallelism() + " threads");
        System.out.println("Total      : " + total);
        System.out.println("Expected   : " + (1_000_000L * 1_000_001L / 2));
        pool.shutdown();
    }
}
```

OUTPUT

```
Parallelism : 2 threads
Total      : 500000500000
Expected   : 500000500000
```


Part 2 - Java 8 New Features

Released in March 2014, Java 8 was the biggest change to the language since generics. It brought functional-style programming through lambdas, functional interfaces, method references and the Streams API, plus a completely new Date and Time API.

01	Default Methods in Interfaces	DefaultMethodsDemo
02	Interface Defaults vs Abstract Classes	-
03	Static Methods in Interfaces	StaticInterfaceMethodsDemo
04	Optional - Dealing with Nulls	OptionalDemo
05	Lambda Expressions	LambdaBasicsDemo
06	Anonymous Inner Class vs Lambda	AnonymousVsLambdaDemo
07	Functional Interfaces	CustomFunctionalInterfaceDemo
08	Predicate	PredicateDemo
09	Function	FunctionDemo
10	UnaryOperator and BinaryOperator	OperatorsDemo
11	Consumer and Supplier	ConsumerSupplierDemo
12	Bi-Functional Interfaces	BiFunctionalDemo
13	Primitive Functional Interfaces	PrimitiveFunctionalDemo
14	Method References	MethodReferencesDemo
15	Constructor References	ConstructorReferencesDemo
16	Streams API - Basics	StreamBasicsDemo
17	Streams API - Operations and Collectors	StreamOperationsDemo
18	Collections vs Streams	-
19	New Date and Time API - Local Types	LocalDateTimeDemo
20	Time Zones, Instant, Duration and Formatting	ZonesAndFormattingDemo
21	CompletableFuture	CompletableFutureDemo
22	Map Enhancements	MapEnhancementsDemo
23	Other Miscellaneous Updates	MiscJava8Demo

2.1 Default Methods in Interfaces

An interface method marked **default** has a body. Implementing classes inherit it and may override it. This allowed the JDK to add methods such as `forEach` and `stream` to existing collection interfaces without breaking millions of implementations.

KEY POINTS

- Default methods are implicitly public.
- If a class inherits the same default method from two interfaces, it must override it; it can call a specific one with `InterfaceName.super.method()`.
- Class methods always win over interface defaults.
- A default method cannot override `equals`, `hashCode` or `toString` from `Object`.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/DefaultMethodsDemo.java

```
/** Java 8: interfaces can contain concrete "default" methods. */
public class DefaultMethodsDemo {

    interface PaymentGateway {
        void pay(double amount);                // abstract, as before

        default double fee(double amount) {      // new, with a body
            return amount * 0.02;
        }
    }

    // Existing implementations keep compiling even though fee() was added later
    static class UpiGateway implements PaymentGateway {
        public void pay(double amount) {
            System.out.println("UPI paid " + amount + ", fee " + fee(amount));
        }
    }

    static class CardGateway implements PaymentGateway {
        public void pay(double amount) {
            System.out.println("Card paid " + amount + ", fee " + fee(amount));
        }
        @Override public double fee(double amount) { return amount * 0.03; } // override
    }

    // Two interfaces with the same default method -> the class MUST resolve it
    interface Printer { default String status() { return "Printer ready"; } }
    interface Scanner { default String status() { return "Scanner ready"; } }

    static class OfficeMachine implements Printer, Scanner {
        @Override public String status() {
            return Printer.super.status() + " & " + Scanner.super.status();
        }
    }

    public static void main(String[] args) {
        new UpiGateway().pay(1000);
        new CardGateway().pay(1000);
        System.out.println(new OfficeMachine().status());
    }
}
```

OUTPUT

```
UPI paid 1000.0, fee 20.0
Card paid 1000.0, fee 30.0
Printer ready & Scanner ready
```

2.2 Interface Defaults vs Abstract Classes

Although interfaces can now contain behaviour, abstract classes still have a distinct role.

KEY POINTS

- Interfaces cannot hold instance state (fields are public static final); abstract classes can.
- Abstract classes have constructors; interfaces do not.
- A class can implement many interfaces but extend only one class.
- Use interfaces for capabilities/contracts, abstract classes for a shared base with state.

2.3 Static Methods in Interfaces

Interfaces can declare **static** methods, so factory and helper methods can live with the type they belong to instead of in a separate utility class.

KEY POINTS

- Called with InterfaceName.method() only.
- Not inherited by implementing classes or sub-interfaces.
- JDK examples: Comparator.comparing, Function.identity, Stream.of.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/StaticInterfaceMethodsDemo.java

```
/** Java 8: interfaces can contain static utility methods. */
public class StaticInterfaceMethodsDemo {

    interface Validator {
        boolean isValid(String input);

        // Factory / helper methods live next to the type they belong to
        static Validator minLength(int length) { return s -> s != null && s.length() >= length; }
        static Validator digitsOnly()          { return s -> s != null && s.matches("\\d+"); }
    }

    static class PanValidator implements Validator {
        public boolean isValid(String s) { return s.matches("[A-Z]{5}[0-9]{4}[A-Z]"); }
        // Static interface methods are NOT inherited: PanValidator.minLength(3) won't compile
    }

    public static void main(String[] args) {
        Validator pin = Validator.digitsOnly();
        Validator password = Validator.minLength(8);

        System.out.println("PIN 560001 valid?      " + pin.isValid("560001"));
        System.out.println("PIN 56A001 valid?      " + pin.isValid("56A001"));
        System.out.println("Password 'secret'?    " + password.isValid("secret"));
        System.out.println("PAN ABCDE1234F valid? " + new PanValidator().isValid("ABCDE1234F"));
    }
}
```

OUTPUT

```
PIN 560001 valid?      true
PIN 56A001 valid?      false
Password 'secret'?     false
PAN ABCDE1234F valid? true
```

2.4 Optional - Dealing with Nulls

Optional<T> is a container that either holds a value or is empty. Returning Optional makes it obvious to the caller that a result may be missing, and its methods let you handle that case without explicit null checks.

KEY POINTS

- Create: Optional.of (non-null), Optional.ofNullable, Optional.empty.
- Transform: map, flatMap, filter.
- Retrieve: orElse, orElseGet (lazy), orElseThrow, ifPresent.
- Avoid calling get() without checking - it throws NoSuchElementException when empty.
- Intended mainly as a return type, not for fields or method parameters.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/OptionalDemo.java

```
/** Java 8: Optional makes "value may be absent" explicit and avoids null checks. */
public class OptionalDemo {

    static class Address {
        private final String city;
        Address(String city) { this.city = city; }
        String getCity() { return city; }
    }

    static class Customer {
        private final String name;
        private final Address address; // may be null
        Customer(String name, Address address) { this.name = name; this.address = address; }
        String getName() { return name; }
        Optional<Address> getAddress() { return Optional.ofNullable(address); }
    }

    static final Map<String, Customer> DB = new HashMap<>();
    static {
        DB.put("C1", new Customer("Asha", new Address("Pune")));
        DB.put("C2", new Customer("Vikram", null));
    }

    static Optional<Customer> findById(String id) { return Optional.ofNullable(DB.get(id)); }

    static String cityOf(String id) {
        return findById(id)
            .flatMap(Customer::getAddress) // Optional<Address>
            .map(Address::getCity)         // Optional<String>
            .orElse("City not available");
    }

    public static void main(String[] args) {
        System.out.println("C1 city : " + cityOf("C1"));
        System.out.println("C2 city : " + cityOf("C2"));
        System.out.println("C9 city : " + cityOf("C9"));

        findById("C1").ifPresent(c -> System.out.println("Found : " + c.getName()));
        System.out.println("C9 present? " + findById("C9").isPresent());

        Optional<String> name = findById("C2").map(Customer::getName)
            .filter(n -> n.startsWith("V"));
        System.out.println("Filtered: " + name.get());

        String fallback = findById("C9").map(Customer::getName).orElseGet(() -> "Guest");
        System.out.println("orElseGet: " + fallback);

        try {
            findById("C9").orElseThrow(() -> new IllegalArgumentException("No customer C9"));
        } catch (IllegalArgumentException e) {
            System.out.println("orElseThrow: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
C1 city : Pune
C2 city : City not available
C9 city : City not available
Found   : Asha
C9 present? false
Filtered: Vikram
orElseGet: Guest
orElseThrow: No customer C9
```

2.5 Lambda Expressions

A lambda is an anonymous function: **(parameters) -> body**. It provides the implementation of a functional interface's single abstract method, so behaviour can be passed around as data.

KEY POINTS

- Parameter types are usually inferred; parentheses are optional for a single parameter.
- A single-expression body needs no braces or return.
- Lambdas can use local variables only if they are effectively final.
- Enable concise code with collections, streams, threads and event handlers.

EXAMPLE PROGRAM

```
java8/src/main/java/com/training/java8/LambdaBasicsDemo.java
```

```
/** Java 8: lambda expressions - the different syntax forms. */
public class LambdaBasicsDemo {

    interface Greeter    { void greet(); }
    interface Notifier   { void send(String message); }
    interface Calculator { int apply(int a, int b); }
    interface TaxCalculator { double tax(double income); }

    public static void main(String[] args) {
        // No parameters
        Greeter greeter = () -> System.out.println("Welcome to Java 8!");

        // One parameter - parentheses optional, type inferred
        Notifier sms = msg -> System.out.println("SMS: " + msg);

        // Two parameters, single expression - 'return' is implicit
        Calculator add = (a, b) -> a + b;
        Calculator max = (int a, int b) -> a > b ? a : b;    // explicit types allowed

        // Block body - braces and explicit 'return' needed
        TaxCalculator tax = income -> {
            if (income <= 700_000) return 0;
            return (income - 700_000) * 0.10;
        };

        greeter.greet();
        sms.send("Your OTP is 4821");
        System.out.println("add(7, 5) = " + add.apply(7, 5));
        System.out.println("max(7, 5) = " + max.apply(7, 5));
        System.out.println("tax(1200000) = " + tax.tax(1_200_000));

        // Lambdas can use local variables that are effectively final
        String bank = "SBI";
        Notifier email = msg -> System.out.println("Email from " + bank + ": " + msg);
        email.send("Statement ready");
    }
}
```

OUTPUT

```
Welcome to Java 8!
SMS: Your OTP is 4821
add(7, 5) = 12
max(7, 5) = 7
tax(1200000) = 50000.0
Email from SBI: Statement ready
```

2.6 Anonymous Inner Class vs Lambda

Lambdas replace most single-method anonymous classes, but the two are not identical.

KEY POINTS

- Inside a lambda, 'this' refers to the enclosing object; in an anonymous class it refers to the anonymous instance.
- Anonymous classes can have state and implement interfaces with several methods.
- Lambdas do not create a separate .class file; they use invokedynamic at runtime.
- Lambdas cannot shadow local variables of the enclosing scope.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/AnonymousVsLambdaDemo.java

```
/** Java 8: the same behaviour with an anonymous inner class and with a lambda. */
public class AnonymousVsLambdaDemo {

    private final String label = "outer";

    void showThis() {
        Runnable anonymous = new Runnable() {
            private final String label = "anonymous";
            public void run() { System.out.println("anonymous 'this.label' = " + this.label); }
        };
        Runnable lambda = () -> System.out.println("lambda 'this.label' = " + this.label);
        anonymous.run(); // 'this' = the anonymous class instance
        lambda.run();   // 'this' = the enclosing object
    }

    public static void main(String[] args) {
        List<String> names = new ArrayList<>(Arrays.asList("Zara", "Bala", "Mohan"));

        // Anonymous inner class - verbose
        Collections.sort(names, new Comparator<String>() {
            @Override public int compare(String a, String b) { return a.compareTo(b); }
        });
        System.out.println("Anonymous sort: " + names);

        // Lambda - concise
        Collections.sort(names, (a, b) -> b.compareTo(a));
        System.out.println("Lambda sort : " + names);

        new AnonymousVsLambdaDemo().showThis();
    }
}
```

OUTPUT

```
Anonymous sort: [Bala, Mohan, Zara]
Lambda sort : [Zara, Mohan, Bala]
anonymous 'this.label' = anonymous
lambda 'this.label' = outer
```

2.7 Functional Interfaces

A functional interface has exactly one abstract method (SAM - Single Abstract Method). Any such interface can be the target of a lambda or method reference. The optional **@FunctionalInterface** annotation makes the compiler enforce the rule.

KEY POINTS

- Default and static methods do not count toward the single abstract method.
- Abstract methods that match public Object methods (toString, equals) do not count either.
- Existing JDK interfaces such as Runnable, Callable and Comparator are functional.
- java.util.function provides 43 ready-made interfaces - covered next.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/CustomFunctionalInterfaceDemo.java

```
/** Java 8: a functional interface has exactly one abstract method. */
public class CustomFunctionalInterfaceDemo {

    @FunctionalInterface          // compiler error if a 2nd abstract method is added
    interface DiscountRule {
        double apply(double price);

        default DiscountRule andThen(DiscountRule next) {    // defaults are allowed
            return price -> next.apply(apply(price));
        }
        static DiscountRule none() { return price -> price; } // statics are allowed
        String toString();    // Object methods don't count as abstract methods
    }

    public static void main(String[] args) {
        DiscountRule festive = price -> price * 0.90;          // 10% off
        DiscountRule coupon  = price -> price - 200;           // flat Rs.200 off

        double cartValue = 5_000;
        System.out.println("No discount   : " + DiscountRule.none().apply(cartValue));
        System.out.println("Festive      : " + festive.apply(cartValue));
        System.out.println("Festive+coupon: " + festive.andThen(coupon).apply(cartValue));
    }
}
```

OUTPUT

```
No discount   : 5000.0
Festive      : 4500.0
Festive+coupon: 4300.0
```

2.8 Predicate

Predicate<T> - **boolean test(T t)**. Represents a condition. Used by `filter()`, `removeIf()` and `anyMatch()`.

KEY POINTS

- Combine with `and()`, `or()`, `negate()`.
- `Predicate.isEqual(x)` creates an equality check; Java 11 adds `Predicate.not()`.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/PredicateDemo.java

```
/** Java 8: Predicate<T> takes a value and returns boolean. */
public class PredicateDemo {

    public static void main(String[] args) {
        Predicate<Employee> inIT      = e -> e.getDepartment().equals("IT");
        Predicate<Employee> highEarner = e -> e.getSalary() > 90_000;
        Predicate<Employee> inBangalore = e -> e.getCity().equals("Bangalore");

        for (Employee e : Employee.sample()) {
            System.out.printf("%-8s IT&high=%-5b IT|BLR=%-5b notIT=%b%n",
                              e.getName(),
                              inIT.and(highEarner).test(e),
                              inIT.or(inBangalore).test(e),
                              inIT.negate().test(e));
        }

        Predicate<String> isAdmin = Predicate.isEqual("admin");
        System.out.println("isEqual(\"admin\") on \"admin\": " + isAdmin.test("admin"));
    }
}
```

OUTPUT

```
Arun      IT&high=true IT|BLR=true notIT=false
Priya     IT&high=false IT|BLR=false notIT=true
Karthik   IT&high=true IT|BLR=true notIT=false
Meena     IT&high=false IT|BLR=true notIT=true
Rahul     IT&high=false IT|BLR=false notIT=true
Divya     IT&high=false IT|BLR=true notIT=false
isEqual("admin") on "admin": true
```

2.9 Function

Function<T, R> - **R apply(T t)**. Converts one value into another. Used by `map()`, `computeIfAbsent()` and `Comparator.comparing()`.

KEY POINTS

- `f.andThen(g)` applies `f` first, then `g`.
- `f.compose(g)` applies `g` first, then `f`.
- `Function.identity()` returns its input unchanged.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/FunctionDemo.java

```
/** Java 8: Function<T, R> converts a T into an R. */
public class FunctionDemo {

    public static void main(String[] args) {
        Function<Employee, String> toName = Employee::getName;
        Function<String, Integer> length = String::length;
        Function<Double, Double> addBonus = s -> s * 1.10;
        Function<Double, Double> minusTax = s -> s * 0.70;

        Employee arun = Employee.sample().get(0);
        System.out.println("Name          : " + toName.apply(arun));
        System.out.println("Name length    : " + toName.andThen(length).apply(arun));

        // andThen : apply this, then the other      compose : apply the other first
        System.out.println("bonus then tax : " + addBonus.andThen(minusTax).apply(100.0));
        System.out.println("tax composed first: " + addBonus.compose(minusTax).apply(100.0));
        System.out.println("identity      : " + Function.<String>identity().apply("same"));
    }
}
```

OUTPUT

```
Name          : Arun
Name length    : 4
bonus then tax : 77.0
tax composed first: 77.0
identity      : same
```

2.10 UnaryOperator and BinaryOperator

UnaryOperator<T> is a Function whose input and output are the same type. **BinaryOperator<T>** takes two values of type T and returns a T - the shape used by reduce().

KEY POINTS

- UnaryOperator extends Function<T, T>; used by List.replaceAll().
- BinaryOperator extends BiFunction<T, T, T>.
- BinaryOperator.minBy / maxBy build operators from a Comparator.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/OperatorsDemo.java

```
/** Java 8: UnaryOperator<T> and BinaryOperator<T> - input and output are the same type. */
public class OperatorsDemo {

    public static void main(String[] args) {
        // UnaryOperator<T> extends Function<T, T>
        UnaryOperator<String> mask = card -> "XXXX-XXXX-" + card.substring(card.length() - 4);
        UnaryOperator<String> upper = String::toUpperCase;
        System.out.println(mask.apply("4111222233334444"));
        System.out.println(upper.andThen(s -> s + "!").apply("done"));

        // BinaryOperator<T> extends BiFunction<T, T, T>
        BinaryOperator<Integer> add = Integer::sum;
        System.out.println("add(40, 2) = " + add.apply(40, 2));

        Comparator<Employee> bySalary = Comparator.comparingDouble(Employee::getSalary);
        BinaryOperator<Employee> higherPaid = BinaryOperator.maxBy(bySalary);
        BinaryOperator<Employee> lowerPaid = BinaryOperator.minBy(bySalary);
        Employee a = Employee.sample().get(0), b = Employee.sample().get(2);
        System.out.println("maxBy: " + higherPaid.apply(a, b));
        System.out.println("minBy: " + lowerPaid.apply(a, b));
    }
}
```

OUTPUT

```
XXXX-XXXX-4444
DONE!
add(40, 2) = 42
maxBy: Karthik(IT, 120000.0)
minBy: Arun(IT, 95000.0)
```

2.11 Consumer and Supplier

Consumer<T> - **void accept(T t)** - takes a value and performs a side effect. **Supplier<T>** - **T get()** - takes nothing and produces a value.

KEY POINTS

- Consumer is used by `forEach()`; chain with `andThen()`.
- Supplier is used by `Optional.orElseGet()`, `Stream.generate()`, and for lazy creation.
- Output of this example varies (random values), so it prints checks rather than values.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/ConsumerSupplierDemo.java

```
/** Java 8: Consumer<T> accepts a value (no result); Supplier<T> produces a value (no input). */
public class ConsumerSupplierDemo {

    public static void main(String[] args) {
        Consumer<Employee> printName = e -> System.out.print(e.getName());
        Consumer<Employee> printDept = e -> System.out.println(" works in " + e.getDepartment());
        Employee.sample().forEach(printName.andThen(printDept));

        Supplier<String> txnId = () -> "TXN-" + UUID.randomUUID().toString().substring(0, 8);
        Supplier<Integer> otp = () -> 1000 + new Random().nextInt(9000);
        Supplier<LocalTime> now = LocalTime::now;

        System.out.println("Transaction id length: " + txnId.get().length());
        int code = otp.get();
        System.out.println("OTP is 4 digits      : " + (code >= 1000 && code <= 9999));
        System.out.println("Supplier<LocalTime> : " + (now.get() != null));
    }
}
```

OUTPUT

```
Arun works in IT
Priya works in Finance
Karthik works in IT
Meena works in HR
Rahul works in Finance
Divya works in IT
Transaction id length: 12
OTP is 4 digits      : true
Supplier<LocalTime> : true
```

2.12 Bi-Functional Interfaces

Two-argument versions of the core interfaces: **BiPredicate<T, U>**, **BiFunction<T, U, R>** and **BiConsumer<T, U>**.

KEY POINTS

- Map.forEach takes a BiConsumer; Map.compute/merge take a BiFunction.
- BiFunction.andThen(Function) transforms the result.
- There is no BiSupplier - a supplier takes no input.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/BiFunctionalDemo.java

```
/** Java 8: two-argument versions - BiPredicate, BiFunction, BiConsumer. */
public class BiFunctionalDemo {

    public static void main(String[] args) {
        BiPredicate<String, String> login =
            (user, pwd) -> user.equals("admin") && pwd.equals("s3cret");
        System.out.println("admin/s3cret -> " + login.test("admin", "s3cret"));
        System.out.println("admin/1234 -> " + login.test("admin", "1234"));

        BiFunction<Double, Integer, Double> emi = (principal, months) -> {
            double r = 0.10 / 12; // 10% annual interest
            return principal * r * Math.pow(1 + r, months) / (Math.pow(1 + r, months) - 1);
        };
        System.out.printf("EMI for 5,00,000 over 24 months = %.2f%n", emi.apply(500_000.0, 24));

        BiFunction<Double, Integer, Long> rounded = emi.andThen(Math::round);
        System.out.println("Rounded EMI = " + rounded.apply(500_000.0, 24));

        Map<String, Integer> stock = new HashMap<>();
        stock.put("Laptop", 12);
        stock.put("Mouse", 150);
        BiConsumer<String, Integer> report = (item, qty) -> System.out.println(item + " : " + qty);
        stock.forEach(report); // Map.forEach takes a BiConsumer
    }
}
```

OUTPUT

```
admin/s3cret -> true
admin/1234 -> false
EMI for 5,00,000 over 24 months = 23072.46
Rounded EMI = 23072
Laptop : 12
Mouse : 150
```

2.13 Primitive Functional Interfaces

Generic interfaces work only with objects, so an int would be boxed to Integer. Primitive specialisations such as **IntPredicate**, **IntFunction**, **ToIntFunction**, **IntUnaryOperator** and **DoubleBinaryOperator** avoid that boxing cost.

KEY POINTS

- Naming pattern: Int/Long/Double + Predicate/Function/Consumer/Supplier/UnaryOperator/BinaryOperator.
- ToXxxFunction returns a primitive from an object; XxxToYyyFunction converts primitives.
- Method names change: applyAsInt, getAsInt, applyAsDouble, etc.
- Primitive streams (IntStream, LongStream, DoubleStream) use them.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/PrimitiveFunctionalDemo.java

```
/**
 * Java 8: primitive specialisations (IntPredicate, ToIntFunction, ...) avoid
 * boxing int -> Integer, which is faster and creates less garbage.
 */
public class PrimitiveFunctionalDemo {

    public static void main(String[] args) {
        IntPredicate isEven      = n -> n % 2 == 0;
        IntUnaryOperator square   = n -> n * n;
        IntFunction<String> stars = n -> new String(new char[n]).replace('\0', '*');
        IntSupplier diceRoll      = () -> 4;          // chosen by fair dice roll
        ToIntFunction<String> length = String::length;
        ToDoubleFunction<Employee> salary = Employee::getSalary;
        DoubleBinaryOperator percent = (part, total) -> part * 100 / total;
        LongConsumer logger       = v -> System.out.println("LongConsumer got " + v);

        System.out.println("isEven(10)      = " + isEven.test(10));
        System.out.println("square(12)       = " + square.applyAsInt(12));
        System.out.println("square.andThen(+1) = " + square.andThen(n -> n + 1).applyAsInt(3));
        System.out.println("stars(5)         = " + stars.apply(5));
        System.out.println("diceRoll()        = " + diceRoll.getAsInt());
        System.out.println("length(\"Bangalore\") = " + length.applyAsInt("Bangalore"));
        System.out.println("salary(Arun)      = "
            + salary.applyAsDouble(Employee.sample().get(0)));
        System.out.println("percent(45, 60)   = " + percent.applyAsDouble(45, 60));
        logger.accept(9_876_543_210L);
    }
}
```

OUTPUT

```
isEven(10)      = true
square(12)       = 144
square.andThen(+1) = 10
stars(5)         = *****
diceRoll()        = 4
length("Bangalore") = 9
salary(Arun)      = 95000.0
percent(45, 60)   = 75.0
LongConsumer got 9876543210
```

2.14 Method References

When a lambda does nothing except call an existing method, it can be replaced with a method reference using ::.

KEY POINTS

- Static method: `ClassName::staticMethod`.
- Instance method of a particular object: `object::method` (e.g. `System.out::println`).
- Instance method of an arbitrary object: `ClassName::method` (first argument becomes the target).
- Constructor: `ClassName::new`.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/MethodReferencesDemo.java

```
/** Java 8: method references - a shorter form of lambdas that just call a method. */
public class MethodReferencesDemo {

    static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int i = 2; i * i <= n; i++) if (n % i == 0) return false;
        return true;
    }

    private final String prefix = ">> ";
    String decorate(String s) { return prefix + s; }

    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(2, 9, 11, 15, 17);

        // 1. Static method          ClassName::staticMethod
        Function<Integer, Boolean> prime = MethodReferencesDemo::isPrime;
        numbers.forEach(n -> System.out.print(n + (prime.apply(n) ? " prime " : " no ")));
        System.out.println();

        // 2. Instance method of a particular object    object::method
        MethodReferencesDemo demo = new MethodReferencesDemo();
        Function<String, String> deco = demo::decorate;
        System.out.println(deco.apply("decorated"));
        numbers.forEach(System.out::print);           // System.out is an object
        System.out.println();

        // 3. Instance method of an arbitrary object    ClassName::instanceMethod
        Function<String, String> upper = String::toUpperCase;           // s -> s.toUpperCase()
        BiFunction<String, String, Boolean> startsWith = String::startsWith;
        System.out.println(upper.apply("java") + " " + startsWith.apply("Java8", "Java"));

        // 4. Constructor reference    ClassName::new (see ConstructorReferencesDemo)
        Supplier<StringBuilder> sb = StringBuilder::new;
        System.out.println(sb.get().append("built").reverse());
    }
}
```

OUTPUT

```
2 prime 9 no 11 prime 15 no 17 prime
>> decorated
29111517
JAVA true
tliub
```

2.15 Constructor References

ClassName::new refers to a constructor. The compiler chooses the constructor whose parameters match the functional interface.

KEY POINTS

- `Supplier<T>` matches a no-arg constructor; `Function<A, T>` matches a one-arg one.
- Array constructors work too: `int[]::new`.
- Common in streams: `map(Account::new)`, `toCollection(TreeSet::new)`.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/ConstructorReferencesDemo.java

```
/** Java 8: ClassName::new refers to a constructor. */
public class ConstructorReferencesDemo {

    static class Account {
        private final String holder;
        Account() { this("UNKNOWN"); }
        Account(String holder) { this.holder = holder; }
        @Override public String toString() { return "Account[" + holder + "]"; }
    }

    public static void main(String[] args) {
        Supplier<Account> blank = Account::new; // uses Account()
        Function<String, Account> create = Account::new; // uses Account(String)
        System.out.println(blank.get());
        System.out.println(create.apply("Lakshmi"));

        List<Account> accounts = Arrays.asList("Ram", "Sita", "Hari").stream()
            .map(Account::new)
            .collect(Collectors.toList());
        System.out.println(accounts);

        Supplier<List<String>> listFactory = ArrayList::new;
        List<String> list = listFactory.get();
        list.add("created by ArrayList::new");
        System.out.println(list);

        IntFunction<int[]> arrayOf = int[]::new; // array constructor
        System.out.println("int[]::new length = " + arrayOf.apply(5).length);
    }
}
```

OUTPUT

```
Account[UNKNOWN]
Account[Lakshmi]
[Account[Ram], Account[Sita], Account[Hari]]
[created by ArrayList::new]
int[]::new length = 5
```

2.16 Streams API - Basics

A **Stream** is a sequence of elements that supports declarative, pipelined operations. A pipeline has a source, zero or more intermediate operations, and one terminal operation.

KEY POINTS

- Sources: `collection.stream()`, `Arrays.stream`, `Stream.of`, `IntStream.range`, `iterate`, `generate`.
- Intermediate operations are lazy - they run only when a terminal operation is invoked.
- Elements flow through the whole pipeline one at a time (see the filter/map trace).
- A stream cannot be reused once a terminal operation has run.
- Streams do not modify their source.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/StreamBasicsDemo.java

```
/** Java 8: creating streams and the source -> intermediate -> terminal pipeline. */
public class StreamBasicsDemo {

    public static void main(String[] args) {
        // Ways to create a stream
        Stream<String> fromValues = Stream.of("UPI", "Card", "NetBanking");
        Stream<Integer> fromList = Arrays.asList(5, 3, 8).stream();
        IntStream fromArray = Arrays.stream(new int[] { 1, 2, 3 });
        IntStream range = IntStream.rangeClosed(1, 5);
        Stream<Integer> iterate = Stream.iterate(1, n -> n * 2).limit(6);
        Stream<Double> generate = Stream.generate(() -> 0.5).limit(3);

        System.out.println("of      : " + fromValues.collect(Collectors.joining(", ")));
        System.out.println("list     : " + fromList.collect(Collectors.toList()));
        System.out.println("array sum: " + fromArray.sum());
        System.out.println("range    : " + range.boxed().collect(Collectors.toList()));
        System.out.println("iterate  : " + iterate.collect(Collectors.toList()));
        System.out.println("generate : " + generate.collect(Collectors.toList()));

        // Pipelines are lazy: nothing runs until a terminal operation is called
        List<String> names = Arrays.asList("arun", "priya", "karthik", "meena");
        Stream<String> pipeline = names.stream()
            .filter(n -> { System.out.println(" filter " + n); return n.length() > 4; })
            .map(n -> { System.out.println(" map    " + n); return n.toUpperCase(); });
        System.out.println("Pipeline built, nothing printed yet. Running terminal op...");
        System.out.println("Result: " + pipeline.collect(Collectors.toList()));

        // A stream can be consumed only once
        Stream<String> once = names.stream();
        once.count();
        try { once.count(); }
        catch (IllegalStateException e) { System.out.println("Reuse -> " + e.getMessage()); }
    }
}
```

OUTPUT

```
of      : UPI, Card, NetBanking
list     : [5, 3, 8]
array sum: 6
range    : [1, 2, 3, 4, 5]
iterate  : [1, 2, 4, 8, 16, 32]
generate : [0.5, 0.5, 0.5]
Pipeline built, nothing printed yet. Running terminal op...
 filter arun
 filter priya
 map    priya
 filter karthik
 map    karthik
 filter meena
 map    meena
Result: [PRIYA, KARTHIK, MEENA]
Reuse -> stream has already been operated upon or closed
```

2.17 Streams API - Operations and Collectors

The most useful operations for everyday work with collections of data.

KEY POINTS

- Intermediate: filter, map, flatMap, distinct, sorted, limit, skip, peek.
- Terminal: collect, forEach, count, reduce, min/max, anyMatch/allMatch/noneMatch, findFirst.
- Collectors: toList, toSet, toMap, joining, groupingBy, partitioningBy, counting, averagingX, summarizingX, mapping.
- parallelStream() splits work across cores - benefits large, CPU-bound workloads only.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/StreamOperationsDemo.java

```
/** Java 8: common intermediate and terminal stream operations and collectors. */
public class StreamOperationsDemo {

    public static void main(String[] args) {
        List<Employee> staff = Employee.sample();

        List<String> itNames = staff.stream()
            .filter(e -> e.getDepartment().equals("IT"))
            .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
            .map(Employee::getName)
            .collect(Collectors.toList());
        System.out.println("IT staff by salary desc : " + itNames);

        System.out.println("Distinct cities : " + staff.stream()
            .map(Employee::getCity).distinct().collect(Collectors.toList()));
        System.out.println("Top 2 earners : " + staff.stream()
            .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
            .limit(2).map(Employee::getName).collect(Collectors.toList()));
        System.out.println("Skip 4 : " + staff.stream()
            .skip(4).map(Employee::getName).collect(Collectors.toList()));

        double payroll = staff.stream().mapToDouble(Employee::getSalary).sum();
        Optional<Employee> top = staff.stream().max(Comparator.comparingDouble(Employee::getSalary));
        System.out.println("Total payroll : " + payroll);
        System.out.println("Highest paid : " + top.get().getName());
        System.out.println("Anyone in Mumbai? : "
            + staff.stream().anyMatch(e -> e.getCity().equals("Mumbai")));
        System.out.println("All earn > 50k? : "
            + staff.stream().allMatch(e -> e.getSalary() > 50_000));
        System.out.println("Count in Chennai : "
            + staff.stream().filter(e -> e.getCity().equals("Chennai")).count());

        int sumOfSquares = Arrays.asList(1, 2, 3, 4).stream().reduce(0, (a, b) -> a + b * b);
        System.out.println("reduce (sum of squares) : " + sumOfSquares);

        List<String> skills = Arrays.asList(Arrays.asList("Java", "SQL"),
            Arrays.asList("Java", "Python")).stream()
            .flatMap(List::stream).distinct().collect(Collectors.toList());
        System.out.println("flatMap skills : " + skills);

        // Collectors
        Map<String, List<String>> byDept = staff.stream().collect(Collectors.groupingBy(
            Employee::getDepartment, Collectors.mapping(Employee::getName, Collectors.toList())));
        Map<String, Double> avgByDept = staff.stream().collect(Collectors.groupingBy(
            Employee::getDepartment, Collectors.averagingDouble(Employee::getSalary)));
        Map<Boolean, Long> above90k = staff.stream().collect(Collectors.partitioningBy(
            e -> e.getSalary() > 90_000, Collectors.counting()));
        DoubleSummaryStatistics stats = staff.stream()
            .collect(Collectors.summarizingDouble(Employee::getSalary));

        System.out.println("groupingBy department : " + byDept);
        System.out.println("average by department : " + avgByDept);
        System.out.println("partitioningBy > 90k : " + above90k);
        System.out.println("summary min/max/avg : " + stats.getMin() + " / "
            + stats.getMax() + " / " + stats.getAverage());
        System.out.println("joining : " + staff.stream()
            .map(Employee::getName).collect(Collectors.joining(" | ", "[", "]")));

        long parallelCount = staff.parallelStream().filter(e -> e.getSalary() > 80_000).count();
        System.out.println("parallelStream count : " + parallelCount);
    }
}
```

OUTPUT

```
IT staff by salary desc : [Karthik, Arun, Divya]
Distinct cities        : [Bangalore, Chennai, Mumbai]
Top 2 earners          : [Karthik, Rahul]
Skip 4                 : [Rahul, Divya]
Total payroll          : 532000.0
Highest paid           : Karthik
Anyone in Mumbai?      : true
All earn > 50k?        : true
Count in Chennai       : 2
reduce (sum of squares) : 30
flatMap skills          : [Java, SQL, Python]
groupingBy department  : {Finance=[Priya, Rahul], HR=[Meena], IT=[Arun, Karthik, Divya]}
average by department  : {Finance=93500.0, HR=60000.0, IT=95000.0}
partitioningBy > 90k    : {false=3, true=3}
summary min/max/avg     : 60000.0 / 120000.0 / 88666.66666666667
joining                 : [Arun | Priya | Karthik | Meena | Rahul | Divya]
parallelStream count    : 4
```

2.18 Collections vs Streams

Collections and streams solve different problems and are often used together.

KEY POINTS

- A collection stores data; a stream processes data.
- Collections are eagerly built; streams are lazily evaluated.
- A collection can be iterated many times; a stream only once.
- Collections use external iteration (for loops); streams use internal iteration.
- Streams can be infinite (iterate, generate) when limited with limit() or takeWhile().

2.19 New Date and Time API - Local Types

The **java.time** package (inspired by Joda-Time) replaces Date and Calendar. **LocalDate**, **LocalTime** and **LocalDateTime** represent date/time without a time zone.

KEY POINTS

- All classes are immutable and thread-safe - every 'plus' or 'with' returns a new object.
- Months are 1-12 (no more 0-based January).
- Period measures date-based amounts (years, months, days); ChronoUnit.between counts units.
- TemporalAdjusters handle 'last day of month', 'next Monday', etc.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/LocalDateTimeDemo.java

```
/** Java 8: java.time - immutable, thread-safe date and time classes. */
public class LocalDateTimeDemo {

    public static void main(String[] args) {
        // Fixed dates keep the output repeatable; use LocalDate.now() in real code
        LocalDate joining = LocalDate.of(2019, Month.JUNE, 15);
        LocalDate today = LocalDate.of(2026, 9, 27);

        System.out.println("Joining date      : " + joining);
        System.out.println("Day of week      : " + joining.getDayOfWeek());
        System.out.println("Leap year?     : " + today.isLeapYear());
        System.out.println("+90 days probation : " + joining.plusDays(90));
        System.out.println("Last day of month : " + today.with(TemporalAdjusters.lastDayOfMonth()));
        System.out.println("Next Monday      : "
            + today.with(TemporalAdjusters.next(DayOfWeek.MONDAY)));

        Period service = Period.between(joining, today);
        System.out.println("Service      : " + service.getYears() + " years "
            + service.getMonths() + " months " + service.getDays() + " days");
        System.out.println("Days between : " + ChronoUnit.DAYS.between(joining, today));
        System.out.println("isBefore     : " + joining.isBefore(today));

        LocalTime marketOpen = LocalTime.of(9, 15);
        System.out.println("Market open  : " + marketOpen);
        System.out.println("Market close : " + marketOpen.plusHours(6).plusMinutes(15));

        LocalDateTime meeting = LocalDateTime.of(today, LocalTime.of(14, 30));
        System.out.println("Meeting      : " + meeting);
        System.out.println("Rescheduled  : " + meeting.plusWeeks(1).withHour(11));

        // Immutable: plusDays returns a NEW object
        today.plusDays(1);
        System.out.println("today unchanged : " + today);
    }
}
```

OUTPUT

```
Joining date      : 2019-06-15
Day of week      : SATURDAY
Leap year?     : false
+90 days probation : 2019-09-13
Last day of month : 2026-09-30
Next Monday      : 2026-09-28
Service      : 7 years 3 months 12 days
Days between : 2661
isBefore     : true
Market open  : 09:15
Market close : 15:30
Meeting      : 2026-09-27T14:30
Rescheduled  : 2026-10-04T11:30
today unchanged : 2026-09-27
```

2.20 Time Zones, Instant, Duration and Formatting

ZonedDateTime adds a time zone, **Instant** is a machine timestamp on the UTC time-line, **Duration** measures time-based amounts, and **DateTimeFormatter** formats and parses text.

KEY POINTS

- Use region IDs like Asia/Kolkata so daylight-saving rules are applied automatically.
- withZoneSameInstant converts the same moment to another zone.
- DateTimeFormatter is immutable and thread-safe, unlike SimpleDateFormat.
- Store Instant/UTC in databases; convert to a zone only for display.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/ZonesAndFormattingDemo.java

```
/** Java 8: time zones, Instant, Duration and DateTimeFormatter. */
public class ZonesAndFormattingDemo {

    public static void main(String[] args) {
        LocalDateTime callTime = LocalDateTime.of(2026, 9, 27, 19, 30);
        ZonedDateTime bangalore = callTime.atZone(ZoneId.of("Asia/Kolkata"));
        ZonedDateTime newYork = bangalore.withZoneSameInstant(ZoneId.of("America/New_York"));
        ZonedDateTime london = bangalore.withZoneSameInstant(ZoneId.of("Europe/London"));

        System.out.println("Bangalore : " + bangalore);
        System.out.println("New York : " + newYork);
        System.out.println("London : " + london);

        Instant start = Instant.parse("2026-09-27T10:00:00Z"); // machine timestamp (UTC)
        Instant end = start.plusSeconds(5_400);
        Duration batch = Duration.between(start, end);
        System.out.println("Instant : " + start);
        System.out.println("Duration : " + batch + " = " + batch.toMinutes() + " minutes");

        DateTimeFormatter indian = DateTimeFormatter.ofPattern("dd-MM-yyyy");
        DateTimeFormatter pretty = DateTimeFormatter.ofPattern("EEEE, dd MMM yyyy hh:mm a");
        System.out.println("Formatted : " + callTime.format(indian));
        System.out.println("Formatted : " + callTime.format(pretty));
        System.out.println("ISO : " + callTime.format(DateTimeFormatter.ISO_LOCAL_DATE_TIME));

        LocalDate parsed = LocalDate.parse("15-08-2026", indian);
        System.out.println("Parsed : " + parsed + " (" + parsed.getDayOfWeek() + ")");
    }
}
```

OUTPUT

```
Bangalore : 2026-09-27T19:30+05:30[Asia/Kolkata]
New York : 2026-09-27T10:00-04:00[America/New_York]
London : 2026-09-27T15:00+01:00[Europe/London]
Instant : 2026-09-27T10:00:00Z
Duration : PT1H30M = 90 minutes
Formatted : 27-09-2026
Formatted : Sunday, 27 Sep 2026 07:30 PM
ISO : 2026-09-27T19:30:00
Parsed : 2026-08-15 (SATURDAY)
```

2.21 CompletableFuture

CompletableFuture represents a result that will be available later and lets you chain and combine asynchronous steps without blocking threads.

KEY POINTS

- Start: `supplyAsync` (returns a value), `runAsync` (no value).
- Chain: `thenApply` (transform), `thenAccept` (consume), `thenCompose` (next async step).
- Combine: `thenCombine` (two futures), `allOf` / `anyOf` (many futures).
- Errors: exceptionally, `handle`, `whenComplete`.
- Runs on the common `ForkJoinPool` unless you pass your own `Executor`.
- The email/sms lines may print in either order.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/CompletableFutureDemo.java

```
/** Java 8: CompletableFuture composes asynchronous tasks without blocking. */
public class CompletableFutureDemo {

    static void pause(long ms) {
        try { TimeUnit.MILLISECONDS.sleep(ms); } catch (InterruptedException ignored) { }
    }

    static double fetchBalance(String acc) { pause(300); return 25_000; }
    static double fetchCreditLimit(String acc) { pause(400); return 50_000; }

    public static void main(String[] args) {
        long t0 = System.currentTimeMillis();

        CompletableFuture<Double> balance = CompletableFuture.supplyAsync(() -> fetchBalance("A1"));
        CompletableFuture<Double> limit = CompletableFuture.supplyAsync(() -> fetchCreditLimit("A1"));

        // Combine two independent async results
        CompletableFuture<String> summary = balance
            .thenCombine(limit, (b, l) -> b + l) // run when both are done
            .thenApply(total -> "Available funds: " + total); // transform the result
        System.out.println(summary.join());
        long elapsed = System.currentTimeMillis() - t0;
        System.out.println("Ran in parallel (< 650 ms)? " + (elapsed < 650));

        // Chain dependent steps and handle failures
        CompletableFuture.supplyAsync(() -> "ORD-77")
            .thenCompose(id -> CompletableFuture.supplyAsync(() -> id + " -> PAID"))
            .thenAccept(status -> System.out.println("Order status: " + status))
            .join();

        String result = CompletableFuture.supplyAsync(() -> {
            if (true) throw new IllegalStateException("Payment service down");
            return "OK";
        })
            .exceptionally(ex -> "Fallback used: " + ex.getCause().getMessage())
            .join();
        System.out.println(result);

        CompletableFuture<Void> all = CompletableFuture.allOf(
            CompletableFuture.runAsync(() -> System.out.println(" sending email...")),
            CompletableFuture.runAsync(() -> System.out.println(" sending sms...")));
        all.join();
        System.out.println("All notifications sent");
    }
}
```

OUTPUT

```
Available funds: 75000.0
Ran in parallel (< 650 ms)? true
Order status: ORD-77 -> PAID
Fallback used: Payment service down
  sending email...
  sending sms...
All notifications sent
```

2.22 Map Enhancements

Java 8 added many default methods to Map that replace common check-then-act code.

KEY POINTS

- `getOrDefault`, `putIfAbsent`, `remove(key, value)`, `replace(key, old, new)`.
- `compute`, `computeIfAbsent`, `computeIfPresent`, `merge` - `merge` is ideal for counting.
- `forEach(BiConsumer)` and `replaceAll(BiFunction)`.
- `Map.Entry.comparingByKey` / `comparingByValue` for sorting entries.
- Internally, `HashMap` buckets with many collisions now switch to balanced trees.

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/MapEnhancementsDemo.java

```
/** Java 8: new default methods on Map. */
public class MapEnhancementsDemo {

    public static void main(String[] args) {
        Map<String, Integer> stock = new TreeMap<>();
        stock.put("Laptop", 10);
        stock.put("Mouse", 50);

        System.out.println("getOrDefault Keyboard : " + stock.getOrDefault("Keyboard", 0));

        stock.putIfAbsent("Mouse", 999);          // ignored - key exists
        stock.putIfAbsent("Monitor", 8);          // added
        System.out.println("putIfAbsent           : " + stock);

        stock.computeIfAbsent("Webcam", k -> 5);
        stock.computeIfPresent("Laptop", (k, v) -> v - 2);    // sold 2
        stock.compute("Mouse", (k, v) -> v == null ? 1 : v + 25);
        System.out.println("compute*            : " + stock);

        stock.replaceAll((k, v) -> v * 2);
        System.out.println("replaceAll (x2)      : " + stock);

        stock.remove("Webcam", 999);              // only removes if value matches
        stock.replace("Monitor", 16, 20);         // only replaces if old value matches
        System.out.println("remove/replace(k,v)  : " + stock);

        // merge - perfect for counting
        List<String> orders = Arrays.asList("UPI", "Card", "UPI", "Cash", "UPI", "Card");
        Map<String, Integer> countByMode = new TreeMap<>();
        orders.forEach(mode -> countByMode.merge(mode, 1, Integer::sum));
        System.out.println("merge counts        : " + countByMode);

        System.out.print("forEach                : ");
        countByMode.forEach((k, v) -> System.out.print(k + "=" + v + " "));
        System.out.println();
        System.out.println("comparingByValue      : " + countByMode.entrySet().stream()
            .sorted(Map.Entry.comparingByValue()).findFirst().get());
    }
}
```

OUTPUT

```
getOrDefault Keyboard : 0
putIfAbsent           : {Laptop=10, Monitor=8, Mouse=50}
compute*              : {Laptop=8, Monitor=8, Mouse=75, Webcam=5}
replaceAll (x2)        : {Laptop=16, Monitor=16, Mouse=150, Webcam=10}
remove/replace(k,v)    : {Laptop=16, Monitor=20, Mouse=150, Webcam=10}
merge counts           : {Card=2, Cash=1, UPI=3}
forEach                : Card=2 Cash=1 UPI=3
comparingByValue       : Cash=1
```

2.23 Other Miscellaneous Updates

Smaller additions that come up often in real projects.

KEY POINTS

- String.join and StringJoiner for building delimited strings.
- java.util.Base64 encoder/decoder (no more sun.misc classes).
- Collection.removeIf, List.sort, List.replaceAll, Iterable.forEach.
- Comparator.comparing, thenComparing, reversed, nullsFirst.
- Math.addExact / multiplyExact detect overflow; unsigned integer helpers.
- LongAdder / DoubleAdder for high-contention counters.
- Nashorn JavaScript engine and the removal of PermGen (replaced by Metaspace).

EXAMPLE PROGRAM

java8/src/main/java/com/training/java8/MiscJava8Demo.java

```
/** Java 8: smaller but very useful additions. */
public class MiscJava8Demo {

    public static void main(String[] args) {
        // String.join and StringJoiner
        System.out.println("String.join : " + String.join(" > ", "Home", "Accounts", "Loans"));
        StringJoiner csv = new StringJoiner(",", "{", "}");
        csv.add("id").add("name").add("balance");
        System.out.println("StringJoiner: " + csv);

        // Base64 encoder / decoder
        String token = Base64.getEncoder().encodeToString("user:pass".getBytes(StandardCharsets.UTF_8));
        String back = new String(Base64.getDecoder().decode(token), StandardCharsets.UTF_8);
        System.out.println("Base64      : " + token + " -> " + back);

        // Iterable.forEach, Collection.removeIf, List.sort, List.replaceAll
        List<Integer> amounts = new ArrayList<>(Arrays.asList(500, -20, 1500, 0, 300));
        amounts.removeIf(a -> a <= 0);
        amounts.sort(Comparator.reverseOrder());
        amounts.replaceAll(a -> a + 10);
        System.out.print("Collections : ");
        amounts.forEach(a -> System.out.print(a + " "));
        System.out.println();

        // Comparator combinators
        List<Employee> staff = new ArrayList<>(Employee.sample());
        staff.sort(Comparator.comparing(Employee::getDepartment)
            .thenComparing(Employee::getSalary, Comparator.reverseOrder()));
        System.out.print("Comparator  : ");
        staff.forEach(e -> System.out.print(e.getDepartment() + "/" + e.getName() + " "));
        System.out.println();

        // Math exact methods detect overflow
        try { Math.addExact(Integer.MAX_VALUE, 1); }
        catch (ArithmeticException e) { System.out.println("addExact    : " + e.getMessage()); }

        // Unsigned integer helpers
        System.out.println("Unsigned    : " + Integer.toUnsignedString(-1));

        // LongAdder - faster than AtomicLong for high-contention counters
        LongAdder hits = new LongAdder();
        Arrays.asList(1, 2, 3, 4, 5).parallelStream().forEach(i -> hits.increment());
        System.out.println("LongAdder   : " + hits.sum());
    }
}
```

OUTPUT

```
String.join : Home > Accounts > Loans
StringJoiner: {id,name,balance}
Base64      : dXNlcjpwYXNz -> user:pass
Collections : 1510 510 310
Comparator  : Finance/Rahul Finance/Priya HR/Meena IT/Karthik IT/Arun IT/Divya
addExact    : integer overflow
Unsigned    : 4294967295
LongAdder   : 5
```


Part 3 - Java 9 New Features

Released in September 2017, Java 9 is best known for the module system (Project Jigsaw) and JShell. It also refined many Java 8 APIs - streams, Optional, collectors and CompletableFuture - and made immutable collections easy to create.

01	Private Methods in Interfaces	PrivateInterfaceMethodsDemo
02	Try-with-Resources Improvement	TryWithResourcesJava9Demo
03	Collection Factory Methods	CollectionFactoryDemo
04	Stream API Enhancements	StreamJava9Demo
05	Optional Enhancements	OptionalJava9Demo
06	Collectors Enhancements	CollectorsJava9Demo
07	Process API Improvements	ProcessApiDemo
08	Arrays Enhancements	ArraysJava9Demo
09	Other Miscellaneous Updates	MiscJava9Demo

3.1 Private Methods in Interfaces

Interfaces can declare **private** and **private static** methods. Default methods can share common code through them without exposing that code to implementing classes.

KEY POINTS

- Private interface methods must have a body.
- Private static methods can be called from static and default methods; private instance methods only from default/private instance methods.

EXAMPLE PROGRAM

java9/src/main/java/com/training/java9/PrivateInterfaceMethodsDemo.java

```
/** Java 9: interfaces can have private (and private static) helper methods. */
public class PrivateInterfaceMethodsDemo {

    interface ReportGenerator {
        default String dailyReport(String data) { return wrap("DAILY", data); }
        default String monthlyReport(String data) { return wrap("MONTHLY", data); }

        // Shared logic without exposing it to implementing classes
        private String wrap(String type, String data) {
            return header(type) + data + footer();
        }
        private static String header(String type) { return "=== " + type + " ===\n"; }
        private static String footer() { return "\n=== END ==="; }
    }

    public static void main(String[] args) {
        ReportGenerator generator = new ReportGenerator() { };
        System.out.println(generator.dailyReport("Transactions: 1,204"));
        System.out.println(generator.monthlyReport("Transactions: 36,880"));
        // generator.wrap(...) -> compile error: wrap() is private
    }
}
```

OUTPUT

```
=== DAILY ===
Transactions: 1,204
=== END ===
=== MONTHLY ===
Transactions: 36,880
=== END ===
```

3.2 Try-with-Resources Improvement

A resource that is already declared outside the try statement can be used directly, as long as the variable is final or effectively final.

KEY POINTS

- Removes the redundant 'try (Reader r = reader)' re-declaration.
- Several variables can be listed, separated by semicolons.

EXAMPLE PROGRAM

java9/src/main/java/com/training/java9/TryWithResourcesJava9Demo.java

```
/** Java 9: try-with-resources accepts existing effectively-final variables. */
public class TryWithResourcesJava9Demo {

    static String firstLine(BufferedReader reader) throws IOException {
        // Java 7/8 needed: try (BufferedReader r = reader) { ... }
        try (reader) {
            return reader.readLine();
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader config = new BufferedReader(new StringReader("env=prod\nport=8080"));
        BufferedReader users = new BufferedReader(new StringReader("admin\nnguest"));

        System.out.println("First config line: " + firstLine(config));

        try (config; users) { // several variables, separated by ';'
            System.out.println("First user      : " + users.readLine());
        }
        try {
            config.readLine();
        } catch (IOException e) {
            System.out.println("config is closed : " + e.getMessage());
        }
    }
}
```

OUTPUT

```
First config line: env=prod
First user      : admin
config is closed : Stream closed
```

3.3 Collection Factory Methods

List.of, **Set.of**, **Map.of** and **Map.ofEntries** create unmodifiable collections in one line.

KEY POINTS

- The collections are immutable - add/remove/put throw UnsupportedOperationException.
- null elements, keys or values are rejected with NullPointerException.
- Set.of and Map.of reject duplicates with IllegalArgumentException.
- Iteration order of Set.of / Map.of is not guaranteed.

EXAMPLE PROGRAM

java9/src/main/java/com/training/java9/CollectionFactoryDemo.java

```
/** Java 9: List.of, Set.of and Map.of create compact immutable collections. */
public class CollectionFactoryDemo {

    public static void main(String[] args) {
        List<String> modes = List.of("UPI", "Card", "NetBanking");
        Set<Integer> codes = Set.of(200, 404, 500);
        Map<String, Integer> limits = Map.of("UPI", 100_000, "Card", 500_000);
        Map<String, String> branches = Map.ofEntries(
            Map.entry("BLR", "Bangalore"),
            Map.entry("MAA", "Chennai"));

        System.out.println("List.of      : " + modes);
        System.out.println("Set.of size  : " + codes.size() + ", has 404? " + codes.contains(404));
        System.out.println("Map.of UPI   : " + limits.get("UPI"));
        System.out.println("Map.ofEntries: " + branches.get("MAA"));

        try {
            modes.add("Cash");
        } catch (UnsupportedOperationException e) {
            System.out.println("add()      : UnsupportedOperationException (immutable)");
        }
        try {
            List.of("A", null);
        } catch (NullPointerException e) {
            System.out.println("null element : NullPointerException");
        }
        try {
            Set.of("A", "A");
        } catch (IllegalArgumentException e) {
            System.out.println("duplicate   : " + e.getMessage());
        }
    }
}
```

OUTPUT

```
List.of      : [UPI, Card, NetBanking]
Set.of size  : 3, has 404? true
Map.of UPI   : 100000
Map.ofEntries: Chennai
add()      : UnsupportedOperationException (immutable)
null element : NullPointerException
duplicate   : duplicate element: A
```

3.4 Stream API Enhancements

New methods make streams easier to control.

KEY POINTS

- `takeWhile(p)` keeps elements while `p` is true, then stops.
- `dropWhile(p)` skips elements while `p` is true, then keeps the rest.
- Both are designed for ordered streams - compare them with `filter()`.
- `Stream.iterate(seed, hasNext, next)` works like a for loop.
- `Stream.ofNullable(x)` returns an empty stream for null.

EXAMPLE PROGRAM

java9/src/main/java/com/training/java9/StreamJava9Demo.java

```
/** Java 9: takeWhile, dropWhile, iterate with a condition, and ofNullable. */
public class StreamJava9Demo {

    public static void main(String[] args) {
        // Sorted readings - stop as soon as the condition fails
        List<Integer> temps = List.of(21, 24, 27, 31, 29, 22);
        System.out.println("takeWhile < 30 : " + temps.stream()
            .takeWhile(t -> t < 30).collect(Collectors.toList()));
        System.out.println("dropWhile < 30 : " + temps.stream()
            .dropWhile(t -> t < 30).collect(Collectors.toList()));
        System.out.println("filter < 30 : " + temps.stream()
            .filter(t -> t < 30).collect(Collectors.toList()));

        // iterate(seed, hasNext, next) - like a for loop
        System.out.println("iterate : " + Stream.iterate(1, n -> n <= 1000, n -> n * 10)
            .collect(Collectors.toList()));

        // ofNullable - empty stream instead of NullPointerException
        String promoCode = null;
        System.out.println("ofNullable(null) count : " + Stream.ofNullable(promoCode).count());
        System.out.println("ofNullable(\"X\") count : " + Stream.ofNullable("X").count());
    }
}
```

OUTPUT

```
takeWhile < 30 : [21, 24, 27]
dropWhile < 30 : [31, 29, 22]
filter < 30 : [21, 24, 27, 29, 22]
iterate : [1, 10, 100, 1000]
ofNullable(null) count : 0
ofNullable("X") count : 1
```

3.5 Optional Enhancements

Optional gains three methods that remove common workarounds.

KEY POINTS

- `ifPresentOrElse(action, emptyAction)` handles both cases.
- `or(supplier)` returns another `Optional` when empty - builds fallback chains.
- `stream()` turns an `Optional` into a stream of 0 or 1 elements - pairs well with `flatMap`.

EXAMPLE PROGRAM

java9/src/main/java/com/training/java9/OptionalJava9Demo.java

```
/** Java 9: Optional.ifPresentOrElse, or and stream. */
public class OptionalJava9Demo {

    static Optional<String> fromCache(String key) {
        return key.equals("rate") ? Optional.of("7.1% (cache)") : Optional.empty();
    }
    static Optional<String> fromDatabase(String key) {
        return Optional.of(key + " (database)");
    }

    public static void main(String[] args) {
        fromCache("rate").ifPresentOrElse(
            v -> System.out.println("Found    : " + v),
            () -> System.out.println("Missing : rate"));
        fromCache("limit").ifPresentOrElse(
            v -> System.out.println("Found    : " + v),
            () -> System.out.println("Missing : limit"));

        // or() supplies another Optional lazily - great for fallback chains
        System.out.println("or()      : " + fromCache("limit").or(() -> fromDatabase("limit")).get());

        // stream() turns an Optional into a 0- or 1-element stream
        List<String> found = List.of("rate", "limit", "rate").stream()
            .map(OptionalJava9Demo::fromCache)
            .flatMap(Optional::stream)
            .collect(Collectors.toList());
        System.out.println("stream(): " + found);
    }
}
```

OUTPUT

```
Found    : 7.1% (cache)
Missing : limit
or()      : limit (database)
stream(): [7.1% (cache), 7.1% (cache)]
```

3.6 Collectors Enhancements

Collectors.filtering and **Collectors.flatMapping** are designed to be used as downstream collectors inside **groupingBy** or **partitioningBy**.

KEY POINTS

- **filtering** keeps every group in the result even when no element passes - **filter()** before **groupingBy** would drop the group.
- **flatMapping** flattens nested collections per group.

EXAMPLE PROGRAM

java9/src/main/java/com/training/java9/CollectorsJava9Demo.java

```
/** Java 9: Collectors.filtering and Collectors.flatMapping for use inside groupingBy. */
public class CollectorsJava9Demo {

    static class Trainee {
        final String name, batch; final int score; final List<String> skills;
        Trainee(String name, String batch, int score, List<String> skills) {
            this.name = name; this.batch = batch; this.score = score; this.skills = skills;
        }
    }

    public static void main(String[] args) {
        List<Trainee> trainees = List.of(
            new Trainee("Anil", "Java-FSD", 82, List.of("Java", "Spring")),
            new Trainee("Bhavna", "Java-FSD", 58, List.of("Java", "SQL")),
            new Trainee("Chitra", "Python", 91, List.of("Python", "Pandas")),
            new Trainee("Dinesh", "Python", 45, List.of("Python")));

        // filtering keeps every group, even if no element passes the filter
        Map<String, List<String>> passed = trainees.stream().collect(Collectors.groupingBy(
            t -> t.batch, TreeMap::new,
            Collectors.filtering(t -> t.score >= 60,
                Collectors.mapping(t -> t.name, Collectors.toList()))));
        System.out.println("Passed per batch : " + passed);

        Map<String, Set<String>> skills = trainees.stream().collect(Collectors.groupingBy(
            t -> t.batch, TreeMap::new,
            Collectors.flatMapping(t -> t.skills.stream(), Collectors.toCollection(
                java.util.TreeSet::new))));
        System.out.println("Skills per batch : " + skills);
    }
}
```

OUTPUT

```
Passed per batch : {Java-FSD=[Anil], Python=[Chitra]}
Skills per batch : {Java-FSD=[Java, SQL, Spring], Python=[Pandas, Python]}
```

3.7 Process API Improvements

ProcessHandle provides portable access to operating-system processes: the PID, command, start time, parent/children, and completion notifications.

KEY POINTS

- `ProcessHandle.current()` for the running JVM, `allProcesses()` for all visible processes.
- `info()` returns command, arguments, user and CPU time where the OS allows.
- `onExit()` returns a `CompletableFuture` that completes when the process ends.

EXAMPLE PROGRAM

java9/src/main/java/com/training/java9/ProcessApiDemo.java

```
/** Java 9: ProcessHandle gives details about the current and other OS processes. */
public class ProcessApiDemo {

    public static void main(String[] args) throws Exception {
        ProcessHandle self = ProcessHandle.current();
        System.out.println("Current PID > 0      : " + (self.pid() > 0));
        System.out.println("Command available   : " + self.info().command().isPresent());
        System.out.println("Is alive           : " + self.isAlive());

        // Start a child process and react when it finishes
        ProcessBuilder builder = new ProcessBuilder("java", "-version").redirectErrorStream(true);
        builder.redirectOutput(ProcessBuilder.Redirect.DISCARD);
        Process child = builder.start();
        System.out.println("Child parent is me : "
            + child.toHandle().parent().map(p -> p.pid() == self.pid()).orElse(false));
        child.onExit().thenAccept(p -> System.out.println("Child exit code      : "
            + p.exitValue())).join();

        long running = ProcessHandle.allProcesses().count();
        System.out.println("Can list processes : " + (running > 0));
    }
}
```

OUTPUT

```
Current PID > 0      : true
Command available   : true
Is alive           : true
Child parent is me : true
Child exit code      : 0
Can list processes : true
```

3.8 Arrays Enhancements

New methods on **java.util.Arrays** for comparing arrays, with overloads that take ranges.

KEY POINTS

- equals and hashCode with from/to ranges.
- compare returns a lexicographic comparison of two arrays.
- mismatch returns the first differing index, or -1 if the arrays are equal.

EXAMPLE PROGRAM

java9/src/main/java/com/training/java9/ArraysJava9Demo.java

```
/** Java 9: Arrays.equals / compare / mismatch, including range versions. */
public class ArraysJava9Demo {

    public static void main(String[] args) {
        int[] expected = { 10, 20, 30, 40, 50 };
        int[] actual   = { 10, 20, 35, 40, 50 };

        System.out.println("equals           : " + Arrays.equals(expected, actual));
        System.out.println("equals [0..2)    : " + Arrays.equals(expected, 0, 2, actual, 0, 2));
        System.out.println("mismatch index  : " + Arrays.mismatch(expected, actual));
        System.out.println("compare         : " + Arrays.compare(expected, actual)); // -1: 30<35
        System.out.println("compare (strings) : "
            + Arrays.compare(new String[] { "a", "b" }, new String[] { "a", "b" }));
    }
}
```

OUTPUT

```
equals           : false
equals [0..2)    : true
mismatch index   : 2
compare          : -1
compare (strings) : 0
```

3.9 Other Miscellaneous Updates

Smaller language and library improvements.

KEY POINTS

- Diamond operator allowed with anonymous inner classes.
- `@SafeVarargs` allowed on private instance methods.
- `Objects.requireNonNullElse` / `requireNonNullElseGet`.
- `CompletableFuture.completeOnTimeout` and `orTimeout`.
- A single underscore `'_'` is no longer a legal identifier.
- Also new in Java 9: the module system (`module-info.java`), JShell, and G1 as the default GC.

EXAMPLE PROGRAM

java9/src/main/java/com/training/java9/MiscJava9Demo.java

```
/** Java 9: smaller language and API improvements. */
public class MiscJava9Demo {

    // @SafeVarargs is now allowed on private instance methods
    @SafeVarargs
    private <T> int countAll(List<T>... lists) {
        int n = 0;
        for (List<T> l : lists) n += l.size();
        return n;
    }

    public static void main(String[] args) {
        // Diamond operator with anonymous inner classes
        Comparator<String> byLength = new Comparator<>() {
            @Override public int compare(String a, String b) {
                return Integer.compare(a.length(), b.length());
            }
        };
        System.out.println("Diamond + anonymous : " + byLength.compare("Java", "Kotlin"));

        System.out.println("@SafeVarargs private: "
            + new MiscJava9Demo().countAll(List.of(1, 2), List.of(3)));

        // Objects.requireNonNullElse / requireNonNullElseGet
        String region = null;
        System.out.println("requireNonNullElse : " + Objects.requireNonNullElse(region, "SOUTH"));
        System.out.println("requireNonNullElseGet: "
            + Objects.requireNonNullElseGet(region, () -> "DEFAULT"));

        // CompletableFuture timeouts
        String status = CompletableFuture.supplyAsync(() -> {
            try { TimeUnit.SECONDS.sleep(2); } catch (InterruptedException ignored) { }
            return "slow response";
        })
            .completeOnTimeout("timed out - cached value", 300, TimeUnit.MILLISECONDS)
            .join();
        System.out.println("completeOnTimeout : " + status);

        // A single underscore is no longer a valid identifier:
        // int _ = 5; -> compile error from Java 9
        System.out.println("Runtime version : Java " + Runtime.version().major() + " (9+ required)");
    }
}
```

OUTPUT

```
Diamond + anonymous : -1
@SafeVarargs private: 3
requireNonNullElse : SOUTH
requireNonNullElseGet: DEFAULT
completeOnTimeout : timed out - cached value
Runtime version : Java 21 (9+ required)
```